AI-INTEGRATED MULTI-MODAL ACADEMIC EVALUATION, PERFORMANCE INTELLIGENCE, AND AUTOMATED VIVA ASSESSMENT SYSTEM

INDIVIDUAL THESIS

H.I.B Wickramarathne

(IT22239198)

Individual component: Rubric-Scoped Grading of Handwritten Answer Scripts with Language Model Pipeline

Group ID: R26-SE-025

B.Sc. (Hons) Degree in Information Technology Specialized in Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis. Details are provided in Appendix B.

Student name: H.I.B Wickramarathne

Student ID: IT22239198

Signature: _____ Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Ms Sanjeevi Chandrasiri		
Co-supervisor: Ms. Shalini Rupasinghe		

ABSTRACT

Marking large cohorts of handwritten examination scripts is slow and difficult to keep consistent. This individual thesis presents the handwritten grading component of Gradex, a modular AI assessment platform. The component conditions photographed pages, recognises handwriting through a staged commercial OCR policy, segments noisy transcripts into per-question answers, retrieves lecture context restricted to the course being graded, and marks each rubric question through a dual-engine language-model router whose primary engine is a low-rank adapted Llama 3.1 eight-billion-parameter model trained on a balanced synthetic corpus of 1,250 grading examples spanning golden, semantic, partial, confident-incorrect, and invalid responses. Identity assurance, batch orchestration, and a React grading client complete the operational path. Component experiments on synthetic data show exact question attribution under four of six OCR-degradation conditions against a digit-only baseline that fails entirely on three; recovery of twelve of fourteen malformed grading responses with no silently incorrect mark; and rejection of all fifty-four out-of-scope retrieval queries with no cross-course leakage. Manual and synthetic system testing of the deployed pipeline has been completed; formal lecturer feedback is planned. Agreement with human markers on authentic scripts is not claimed and awaits ethical clearance.

Keywords: automated short answer grading, handwriting recognition, retrieval-augmented generation, large language models, educational assessment

ACKNOWLEDGEMENTS

I thank my supervisor and co-supervisor for guidance throughout this research project, and the Faculty of Computing at the Sri Lanka Institute of Information Technology for academic support and infrastructure. I also thank my project teammates for collaboration on the shared Gradex platform. Any remaining errors are my own.

TABLE OF CONTENTS

DECLARATION AND SUPERVISOR CERTIFICATION 2
ABSTRACT 3
ACKNOWLEDGEMENTS 4
TABLE OF CONTENTS 5
LIST OF FIGURES 7
LIST OF TABLES 7
LIST OF ABBREVIATIONS 8
1 INTRODUCTION 9
1.1 Overall Project Context 9
1.2 Individual Component Context 9
1.3 Problem Statement 10
1.4 Research Gap 10
1.5 Aim And Objectives 10
1.6 Research Questions Or Hypotheses 11
1.7 Scope Limitations And Assumptions 11
1.8 Individual Contribution 12
1.9 Thesis Structure 12
2 LITERATURE REVIEW AND RELATED WORK 13
2.1 Review Method And Scope 13
2.2 Domain And Theoretical Foundations 13
2.3 Related Research 14
2.4 Existing Systems Products And Techniques 16
2.5 Gap Synthesis And Conceptual Framework 16
3 RESEARCH METHODOLOGY 18
3.1 Research Design 18
3.2 Data Participants Or Study Materials 18
3.3 Experimental Or Study Procedure 19
3.4 Measures And Evaluation Metrics 20
3.5 Data Analysis 21
3.6 Ethics Privacy Security And Responsible Ai 21
3.7 Reproducibility And Research Artefacts 22
4 PRODUCT AND SOLUTION DESIGN 23
4.1 Requirements 23
4.2 Component Architecture Within The Complete Solution 24

4.3 Design Alternatives And Decisions 25
4.4 Detailed Design 25
4.5 Standards And Professional Practice 28
5 IMPLEMENTATION INTEGRATION AND TESTING 29
5.1 Development Environment And Technology Stack 29
5.2 Component Implementation 29
5.3 Integration With The Other Components 31
5.4 Verification And Software Testing 31
5.5 Deployment And Operation 32
6 RESULTS AND VALIDATION 34
6.1 Research Results 34
6.2 Product And Technical Results 39
6.3 User Or Stakeholder Validation 40
6.4 Integration Results 40
6.5 Summary Of Findings 40
7 DISCUSSION 41
7.1 Interpretation Of Findings 41
7.2 Comparison With Literature Baselines And Existing Solutions 42
7.3 Research Contribution 42
7.4 Product Contribution 43
7.5 Limitations And Threats To Validity 43
7.6 Implications 44
8 COMMERCIALISATION AND SUSTAINABILITY 45
8.1 Value Proposition And Target Users ..... **Error! Bookmark not defined.**
8.2 Market And Competitive Position ..... **Error! Bookmark not defined.**
8.3 Business Or Sustainability Model ..... **Error! Bookmark not defined.**
8.4 Scalability Intellectual Property And Compliance ..... **Error! Bookmark not defined.**
8.5 Contribution To The Sustainable Development Goals ..... **Error! Bookmark not defined.**
9 CONCLUSION AND FUTURE DIRECTIONS 45
9.1 Conclusion 45
9.2 Achievement Of Objectives 45
9.3 Individual Research And Product Contributions 46

9.4 Future Directions 46
REFERENCES 48
GLOSSARY 50
APPENDICES 51
Appendix A Research Instruments And Extended Results 51
Appendix B Academic Integrity And Ai Use Disclosure 51

LIST OF FIGURES

Figure 1 : Training corpus composition by response class 22
Figure 2 : Composition of the 1250-record training corpus by answer class 30
Figure 3 : First segmentation score of the proposed splitter by condition (percent) . 35
Figure 4 : First segmentation score of the baseline by condition (percent) 35
Figure 5 : Normalisation outcomes on 14 marker shapes 37
Figure 6 : Share of retrieval queries behaving as expected by query set (percent) 38
Figure 7 : Mean assigned score by answer class in the training corpus 39

LIST OF TABLES

Table 1: Abbreviations used in this thesis 8
Table 2 : Components of the Gradex platform 9
Table 3 : Comparison of representative approaches 16
Table 4 : Answer classes in the synthetic corpus 18
Table 5 : Model and training configuration 20
Table 6 : Metrics by research question 20
Table 7 : Functional requirements 23
Table 8 : Non-functional requirements 24
Table 9 : Design decisions and rejected alternatives 25
Table 10 : Technology stack of the grading component 29
Table 11 : Automated test modules 31
Table 12 : Status of evaluation activities 34
Table 13 : Segmentation results, 200 synthetic scripts per condition (percent, first and second score) 34
Table 14 : Marker-form resolution 36
Table 15 : Normalisation outcomes on 14 marker shapes 36
Table 16 : Retrieval scoping results 37
Table 17 : Training corpus composition and mean assigned score 38
Table 18 : Positioning against alternative approaches **Error! Bookmark not defined.**
Table 19 : Mapping of the project to the Sustainable Development Goals ..... **Error! Bookmark not defined.**
Table 20 : Synthetic training corpus files 51

LIST OF ABBREVIATIONS

Table 1: Abbreviations used in this thesis

Abbreviation	Meaning
ASAG	Automatic Short Answer Grading
API	Application Programming Interface
ETA	Estimated Time of Arrival / Completion
LLM	Large Language Model
LoRA	Low-Rank Adaptation
OCR	Optical Character Recognition
RAG	Retrieval-Augmented Generation
SDG	Sustainable Development Goal
UI	User Interface

1 INTRODUCTION

1.1 Overall Project Context

Assessment is one of the most labour-intensive responsibilities in higher education. A lecturer must mark every script against a rubric, record marks and return feedback quickly. Marking effort grows with cohort size while lecturer time does not, leading to delayed feedback and fatigue-related inconsistency.

Automatic Short Answer Grading (ASAG) addresses this task. ASAG research has moved from concept mapping to machine learning [1] and from lexical overlap to neural representations [2], [11], [12]. Large language models (LLMs) built on the Transformer [14] can interpret rubrics and justify scores [3], [36], [37], but off-the-shelf models are inconsistent and prompt-sensitive [4], [19].

Gradex (project code R26-SE-025) is a modular AI-assisted assessment platform developed as a group research project: four assessment engines behind a shared server (Gradex_AI_Server) and a shared web client (Gradex_AI_Client), as shown in Table 1.1.

Table 2 : Components of the Gradex platform

Component	Purpose	Relation to this thesis
GradingEngine, grading client UI, fine-tuned Llama 3.1 8B LoRA model	Handwritten answer grading, image to rubric-based marks	Subject of this thesis
DiagramEvaluationEngine	Grading of student-drawn diagrams	Teammate component; out of scope
VivaEvaluationEngine	Viva examination evaluation	Teammate component; out of scope
V2_QuestionExamPredictionEngine	Analytics and question or examination generation	Teammate component; out of scope

1.2 Individual Component Context

This thesis reports one component of Gradex: the handwritten grading engine (GradingEngine), the grading pages of the shared client, and a Llama 3.1 8B model adapted with Low-Rank Adaptation (LoRA) for rubric-aware grading. The engine, mounted at /api/grading, accepts an image of a script, conditions it, extracts text by OCR, splits the transcript into per-question answers, retrieves course material, grades each answer against the lecturer's rubric, validates the student's identity against a

roster and presents results for lecturer review. Software Architecture is the case domain.

1.3 Problem Statement

Manual marking of handwritten short-answer scripts is slow, and the interpretation of "partially correct" varies between markers and over a long session. General-purpose LLMs used through a prompt alone may reward fluent but incorrect answers, penalise correct answers in everyday language, be steered by instructions embedded in student text, and produce ungrounded justifications, a form of hallucination [23]. The input is a handwritten script whose skew, shadows and low contrast degrade OCR, and whose transcript must still be divided into per-question answers.

The research problem is how to design, build and evaluate a grading engine that converts handwritten scripts into per-question text, grades each answer against a lecturer-defined rubric using an adaptable LLM grounded in course-scoped material, and presents results that support review, roster validation and batch operation.

1.4 Research Gap

Chapter 2 reveals four gaps. First, research streams are separated: ASAG studies assume digital text [1], [2], while handwriting recognition studies report error rates without examining their effect on grading [9], [27], [29]. Second, LoRA [6] and QLoRA [7] make it feasible to adapt an 8-billion-parameter model on one accelerator, yet little documented practice exists on building training data for the cases lecturers meet: confident but wrong answers, layman wording, incomplete answers and adversarial input. Third, Retrieval-Augmented Generation (RAG) [5], [32] reduces unsupported output in question answering, but its use for rubric-based grading, with retrieval restricted to one course, is less documented. Fourth, studies seldom describe roster validation, batch jobs with progress, or degradation when the primary model service is unavailable, which decide whether a prototype is usable.

1.5 Aim And Objectives

The aim is to design, implement and evaluate an end-to-end handwritten short-answer grading engine, with its client interface and a fine-tuned LoRA grading

model, that supports lecturers in marking against a rubric while keeping them in control of the final result. The specific objectives are:

1. To review the literature and identify the gaps in Section 1.4.

2. To implement OpenCV image conditioning, OCR with escalation, and an answer splitter aligned to rubric question numbers.

3. To construct a synthetic Alpaca-style dataset of five answer classes and fine-tune Llama 3.1 8B (4-bit) with LoRA (rank 16, scaling factor 16) using Unsloth.

4. To build course-scoped retrieval with ChromaDB and all-MiniLM-L6-v2.

5. To implement per-question grading with a primary and a fallback engine, roster validation, asynchronous batch jobs with ETA polling, and the grading interface.

6. To evaluate the stages and the system through stage-level experiments, automated tests and manual and synthetic testing, and to plan a formal lecturer evaluation.

1.6 Research Questions Or Hypotheses

RQ1. How can image conditioning, OCR with escalation and answer segmentation be arranged so that handwritten scripts yield per-question text of sufficient quality for grading?

RQ2. Can a synthetic, class-balanced instruction dataset be used to fine-tune Llama 3.1 8B with LoRA into a rubric-aware short-answer grader for Software Architecture?

RQ3. Does course-scoped retrieval provide relevant, in-scope context without introducing material from other courses?

RQ4. Which design choices (per-question grading, fallback engines, roster validation, batch jobs with progress polling) are needed to make the engine usable in a realistic lecturer workflow?

1.7 Scope Limitations And Assumptions

Inclusions. Handwritten scripts supplied as images; conditioning; OCR; answer splitting; rubric-based grading of text answers; course-scoped retrieval; LoRA fine-tuning of one base model; roster validation; batch processing; and the

lecturer review interface. Exclusions. The diagram, viva and analytics engines of Gradex, and the grading of diagrams, mathematics and code.

Data. The fine-tuning corpus is entirely synthetic and covers Software Architecture. Ethical clearance to collect real student scripts is being sought, so no real scripts have been used for training or evaluation. The results are therefore evidence of technical feasibility, not an estimate of agreement with human markers.

Assumptions. Answers are in English; recognition quality depends on legibility and image quality; and the system is a decision-support tool in which the lecturer reviews and may change every mark.

1.8 Individual Contribution

The individual contributions are: (a) the handwritten grading pipeline (conditioning, OCR escalation, answer splitter, per-question grading); (b) the five-class synthetic dataset, the LoRA fine-tuning of Llama 3.1 8B and its integration as primary engine with a Groq-hosted fallback; (c) course-scoped retrieval; (d) roster validation, batch jobs with ETA polling and the grading interface; and (e) the stage-level experiments and tests reported in later chapters.

1.9 Thesis Structure

Chapter 2 reviews the literature. Chapter 3 presents the research methodology. Chapter 4 presents the design, Chapter 5 the implementation and testing, Chapter 6 the results, Chapter 7 the discussion, Chapter 8 commercialisation and sustainability, and Chapter 9 the conclusions and future work.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review Method And Scope

This chapter reviews short answer grading, LLMs as graders, handwritten script digitisation, retrieval-augmented generation, parameter-efficient fine-tuning and synthetic instruction data, then compares approaches and states the gap this work addresses.

2.2 Domain And Theoretical Foundations

2.2.1 Early and feature-based approaches

ASAG is the automatic assignment of a grade to a free-text answer of a few words to a few sentences, usually by comparison with a reference answer or rubric. Burrows, Gurevych and Stein organised the field into five eras: concept mapping, information extraction, corpus-based methods, machine learning and evaluation [1]. Early systems searched for manually specified concepts, needing expert effort for every question. Similarity-based methods reduced that effort. Mohler, Bunescu and Mihalcea combined semantic similarity measures with a machine-learning stage on a computer science dataset [11], and Sultan, Salazar and Sumner combined alignment features with sentence embeddings [12].

2.2.2 Neural approaches

Pre-trained encoders such as BERT [13] allowed answer and reference to be encoded jointly and fine-tuned for scoring, and Sentence-BERT [16] made embeddings cheap to compare. Haller and colleagues found that neural models generally outperform feature-based ones, but depend on question-specific training data, generalise poorly across questions and domains, and return a score without reasoning [2]. Essay-scoring reviews stress fairness, transparency and vulnerability to gaming [38], which apply equally to short answers.

2.4.1 Image conditioning

Photographs of scripts exhibit rotation, shadows, uneven illumination and compression artefacts. Classical techniques in OpenCV [25] mitigate these: grayscale conversion and denoising reduce irrelevant variation, and binarisation separates ink from paper. Otsu's method chooses a global threshold by maximising between-class

variance [10] and suits even illumination, whereas local methods such as Sauvola's suit varying backgrounds [26].

2.4.2 Handwriting recognition

Handwriting recognition has progressed from hidden Markov models to recurrent networks [27], [29], with the IAM database supporting comparison [28]. TrOCR pairs pre-trained image and text Transformers and reports strong handwritten and printed text results [9]. Tesseract is a general-purpose baseline, less reliable on unconstrained handwriting [24]. Gradex uses the hosted OCR.space service and handles its limits by escalation.

2.4.3 Segmenting answers

Mapping a free-form transcript onto the question structure of an exam is less covered than page- or line-level layout. Students write question numbers in varied forms ("1.", "Q1", "Question 01", "(a)") and OCR may corrupt markers. Rule-based splitting is transparent and testable but must degrade safely when markers are missing.

2.3 Related Research

2.3.1 Capabilities

Large language models follow natural-language instructions and learn tasks from a few examples in the prompt [15], and instruction tuning improves this [22]. A rubric and a student answer can therefore be placed in a prompt and the model asked for a score and justification. Kortemeyer found that a GPT-4 class model graded many physics solutions broadly consistently with a rubric, but with deviations needing instructor oversight [3]. Mizumoto and Eguchi reported a meaningful relationship between GPT and human essay ratings [37].

2.3.2 Limitations

Chamieh, Zesch and Giebermann found zero-shot and few-shot LLM scoring promising because it needs little task data, but with lower agreement than fine-tuned task-specific models and with variation across prompts and datasets [4]. Zheng and colleagues identified biases of LLM judges, such as a preference for longer answers, while showing that strong models can approximate human preferences [19]. Identical

answers should receive identical marks, and decoding strategy strongly affects generated text [40]; greedy decoding gives repeatability, though not correctness.

2.3.3 Adversarial and invalid input

Because student text enters the prompt, it can contain instructions to the model (for example, a request to award full marks) or be empty or off-topic; judges are susceptible to such manipulation [19]. A grader should meet such input during adaptation and assign low or zero marks predictably, which motivates the adversarial class in the dataset of Chapter 3.

Lewis and colleagues combined a parametric language model with a document memory retrieved at inference time, improving factual grounding without retraining [5]; Gao and colleagues survey its forms [32]. Retrieval can be sparse, as in BM25 [34], or dense [33], using embeddings from models such as Sentence-BERT [16] and MiniLM [17], with efficient similarity search [18]. ChromaDB is an open-source embedding store with metadata filtering, which allows retrieval to be restricted to a subset of documents.

In grading, retrieved material would be the lecturer's notes or slides. Two risks arise: irrelevant passages that mislead the model, and leakage between courses, where a query retrieves content that was never taught. Scoping retrieval by course addresses the second.

Full fine-tuning of an 8-billion-parameter model exceeds the memory of a single commodity GPU. Parameter-efficient methods freeze the base model and train few parameters: adapters insert small trainable modules [30], prompt tuning learns continuous prompts [31], and LoRA represents a weight update as the product of two low-rank matrices, adding no inference latency once merged [6]. QLoRA quantises the base model to 4 bits and trains LoRA adapters through it, enabling single-GPU fine-tuning [7]. Llama 3 provides open-weight bases including an 8-billion-parameter variant [8], and Transformers [39] and Unsloth make the procedure accessible in cloud notebooks.

Instruction-following models are commonly trained on records of instruction, input and output; Alpaca popularised this format with model-generated examples [20], and Self-Instruct showed that models can bootstrap diverse instructions [21]. Synthetic data suit grading because real scripts need ethical clearance and the

designer can ensure that rare but important cases, such as adversarial answers, appear often enough. The risks are that synthetic answers may not capture real errors and that evaluating on data generated like the training data can overstate performance.

2.4 Existing Systems Products And Techniques

Table 3 : Comparison of representative approaches

Approach	Input	Grading mechanism	Grounding	Handwriting handled
Mohler et al. [11]	Digital text	Semantic similarity plus machine learning	Reference answers	No
Sultan et al. [12]	Digital text	Alignment and embedding features	Reference answers	No
Neural graders surveyed in [2]	Digital text	Supervised classification or regression	Per-question training data	No
Zero-shot and few-shot LLMs [3], [4]	Digital text	Prompted LLM	Prompt only	No
This work (GradingEngine)	Photographed or scanned script	Fine-tuned Llama 3.1 8B LoRA with hosted fallback	Course-scoped retrieval	Yes: conditioning, OCR escalation, splitting

The surveyed methods address scoring of text, whereas this work addresses the chain from image to mark. It claims no higher agreement with human graders than those studies, since real-script evaluation has not been carried out.

2.5 Gap Synthesis And Conceptual Framework

Reliability requires identical input to give identical output, motivating deterministic decoding [40]. Validity motivates rubric-based prompts and course-grounded retrieval [5], [32]. Fairness requires that students are not penalised for wording, motivating a class of correct answers in everyday language. Transparency motivates a justification with each score [3], [36], and accountability requires that a human has the final say [36]. Scripts and rosters are also personal data, and sending them to external OCR or LLM services has data-protection implications [36].

ASAG and LLM-grading studies [1]-[4], [37] concentrate on scoring digital text; handwriting studies [9], [27]-[29] on recognition accuracy; RAG studies [5], [32] on question answering; and fine-tuning studies [6], [7] on training methods. Their integration into a lecturer workflow, with data designed around answer

behaviours, course-confined retrieval, fallback behaviour and roster validation, has not been documented in a form that can be examined and repeated. This thesis responds with such an integration for Software Architecture.

3 RESEARCH METHODOLOGY

3.1 Research Design

This chapter sets out the research design, the data and materials, the experimental procedures, the evaluation metrics, the analysis approach, the ethical position and the measures taken for reproducibility. The detailed design of each component is presented in Chapter 4; only a brief overview is given here.

The work follows a design science approach [35], in which an artefact is built to solve an identified problem and then evaluated for utility. The artefact is the grading engine with its client and fine-tuned model, developed iteratively.

Evaluation is organised stage by stage, because in a pipeline an end-to-end score cannot show where an error originated. Each research question from Section 1.6 is paired with an evidence source: RQ1 with segmentation and text normalisation experiments, RQ2 with the dataset design, fine-tuning and manual synthetic testing, RQ3 with a retrieval scoping experiment, and RQ4 with automated test suites and manual workflow testing.

The engine is a service mounted at /api/grading with pages in the shared client, built as a sequence of independently testable stages described in Chapter 4: script upload, OpenCV conditioning, OCR.space with escalation, answer splitting, course-scoped retrieval, per-question grading (primary fine-tuned Llama 3.1 8B LoRA via Colab and ngrok; Groq fallback), roster validation and lecturer review, all surrounded by a batch job manager with ETA polling.

3.2 Data Participants Or Study Materials

3.4.1 Synthetic training corpus

The training corpus is a synthetic dataset of 1250 records in five classes of 250, shuffled before use. All records concern Software Architecture. Classes are defined by answer behaviour rather than topic (Table 3.1).

Table 4 : Answer classes in the synthetic corpus

Class	Records	Description
golden_high_score	250	Accurate, complete answers deserving a high score
semantic_layman	250	Correct answers in everyday rather than technical language

partial_incomplete	250	Partly correct answers deserving partial credit
concept_mixer_confident_incorrect	250	Fluent, confident answers that mix concepts and are incorrect
adversarial_zero	250	Empty, off-topic, invalid or grader-manipulating input deserving zero

Each record follows the Alpaca-style format [20] in JSONL: the instruction states the task and response format, the input holds the question, rubric information and student answer, and the output holds the score and justification.

3.4.2 Stage-level test material

The segmentation experiment used the inputs of the project's ICAC paper, reported as published. The normalisation experiment used 14 test cases containing recognition-style errors, the retrieval experiment 18 retrieval checks, and system-level testing synthetic scripts representing the five answer classes.

3.3 Experimental Or Study Procedure

3.5.1 Conditioning, ocr and segmentation (rq1)

Scripts are conditioned with OpenCV [25] (grayscale conversion, denoising, contrast adjustment, deskewing, optional binarisation [10], [26]) and passed to the hosted OCR.space service, with escalation to alternative candidates, including the unfiltered original image, when a result is unusable. The transcript is split into per-question answers by rule-based marker detection; when no markers are found the whole transcript is passed on rather than a guessed segment. The ICAC segmentation experiment examined the effect of conditioning and splitting, and its results are reproduced without alteration.

3.5.2 Text normalisation (rq1)

The 14 test cases were passed through the normalisation step, which corrects character confusions that obscure question markers; each output was compared with the intended text.

3.5.3 Retrieval scoping (rq3)

Lecturer material is embedded with all-MiniLM-L6-v2 [16], [17] and stored in ChromaDB with course metadata. For each of the 18 checks, retrieval was filtered

by the course identifier. Each returned passage was judged in scope or not, and results from a different course were counted separately.

3.5.4 Fine-tuning (rq2)

Llama 3.1 8B [8] was loaded in 4-bit precision following QLoRA [7] and adapted with LoRA [6] in Google Colab using Unsloth, training only the low-rank adapters on the output portion of each record (Table 3.2). The adapter was then loaded into an inference notebook that exposes an HTTP endpoint.

Table 5 : Model and training configuration

Item	Value
Base model	Llama 3.1 8B
Base weight precision	4-bit quantisation
Adaptation method	LoRA
LoRA rank r	16
LoRA scaling factor alpha	16
Training toolkit	Unsloth
Training environment	Google Colab
Corpus	1250 synthetic records, 5 classes of 250
Inference decoding	Greedy, temperature 0

Inference uses greedy decoding at temperature 0, so outputs are repeatable for a given prompt and model state [40]. A Groq-hosted model is the fallback route, and a flagged placeholder is returned if both routes fail.

3.5.5 Automated and manual testing (rq4)

Automated tests exercise the splitter, roster validation and health checking of the model service. Manual testing followed the whole workflow through the client with synthetic scripts of each answer class, including an unreachable primary model.

3.4 Measures And Evaluation Metrics

Table 6 : Metrics by research question

RQ	Evidence	Metric
RQ1	Segmentation experiment (ICAC paper)	Metrics as published
RQ1	Text normalisation, 14 cases	Cases recovered to intended text, proportion
RQ2	Dataset design, fine-tuning, manual synthetic testing	Qualitative behaviour per answer class; no accuracy claimed
RQ3	Retrieval scoping, 18 checks	In-scope results, results from foreign courses, in-scope proportion

RQ4	Automated tests and manual workflow testing	Tests passing; workflow completion, fallback and flag behaviour
Planned	Lecturer evaluation	Mean absolute error, quadratic weighted kappa, questionnaire ratings

For the stage-level experiments the metric is a simple proportion. For the planned lecturer evaluation, agreement will be measured by mean absolute error and quadratic weighted kappa, which suits ordinal marks.

3.5 Data Analysis

Results are analysed descriptively. Because the normalisation and retrieval samples are small, counts and proportions are reported without inferential statistics, and one case in 14 changes the proportion by more than seven percentage points. Results are interpreted only within the conditions tested.

Construct validity is limited because synthetic answers may not represent real responses. Internal validity is limited by dependence on external services (OCR.space, Groq). External validity is limited by the single domain and the absence of real handwriting variety.

3.6 Ethics Privacy Security And Responsible Ai

All data used so far are synthetic. Ethical clearance to collect real scripts is pending, and real scripts will not be used until it is granted; they will then be anonymised where possible, access restricted, and lecturer participation voluntary. OCR.space and Groq are external services, so data sent to them leave the local environment, which must be resolved in the data-protection arrangements for any real-data study.

A formal evaluation is planned and subject to ethical clearance; no results are reported. Lecturers would mark a sample of scripts independently, and their marks would be compared with the system's using the metrics above. The fine-tuned, base and fallback models would be compared on identical inputs, and lecturers would complete a questionnaire on usability, trust and time saved.

3.7 Reproducibility And Research Artefacts

Class definitions, record counts and the record format are specified (Section 3.4.1), and the model, quantisation, LoRA settings and toolkit are stated (Table 3.2). Inference is deterministic through greedy decoding, and the automated test suites can be re-run.

Chapter 4 presents the system design.

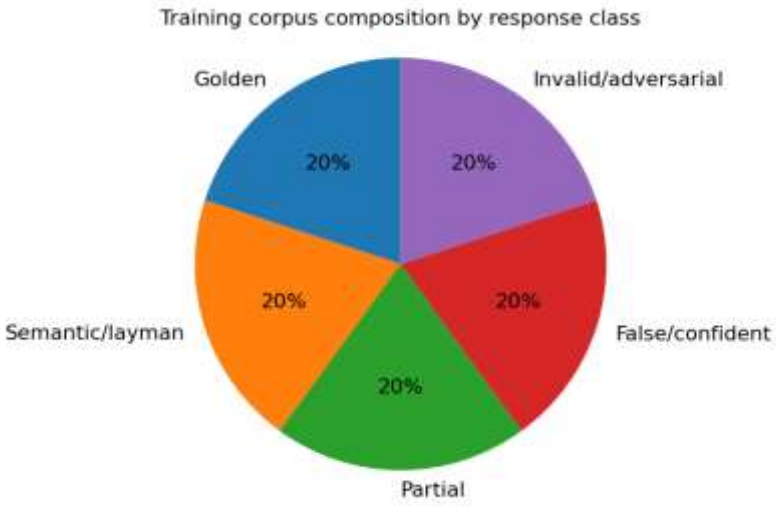


Figure 1 : Training corpus composition by response class

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

4.1.1 Stakeholders and context of use

The primary user is a lecturer with a batch of handwritten scripts, usually phone photographs or scanned PDFs, who must produce a mark and justification for every question. Images are compressed, rotated or shadowed; students label questions inconsistently ("Q1", "Question 01", "1)", "Q I"); OCR may read the Roman numeral I as l or a vertical bar; and the model is hosted on a free notebook platform that can disappear mid-batch. The system must not silently mark the wrong student or abandon a batch because one service failed.

4.1.2 Functional requirements

Table 7 : Functional requirements

ID	Requirement	Realised by
FR1	Accept images or PDF and extract text from every page, even when the first attempt fails	OCR cascade (4.4.1)
FR2	Align answers with rubric question numbers, tolerating numeral, Roman and glyph variants	Answer splitter (4.4.2)
FR3	Never assign text to a question without evidence; report missing answers	Source labels split, full, empty
FR4	Retrieve lecturer material restricted to the selected course	Course-scoped RAG (4.4.3)
FR5	Grade each question separately with score, justification and feedback	Per-question loop
FR6	Fine-tuned primary grader, hosted fallback, last-resort response	Dual-engine router (4.4.4)
FR7	Validate script identity against a roster and block ambiguous cases	Roster validation (4.4.6)
FR8	Grade a class asynchronously with progress and ETA	Batch jobs (4.4.7)
FR9	Review text, context and justification; override and regrade	GradingPage (4.4.8)
FR10	Manage courses and lecture materials; persist rubrics and results	Knowledge Base, MongoDB

4.1.3 Non-functional requirements and constraints

Table 8 : Non-functional requirements

ID	Quality	Requirement
NFR1	Reliability	One failed service call never aborts a batch
NFR2	Safety of marks	Prefer a flagged zero to a guess when evidence is insufficient
NFR3	Transparency	Every mark shows text, engine and retrieval context
NFR4	Responsiveness	Long operations do not hold an HTTP request open
NFR5	Determinism	Greedy decoding gives repeatable output
NFR6	Modularity	Each stage is testable alone and mounts under a route prefix
NFR7	Privacy	No real student data leaves the controlled environment before clearance

Constraints: no GPU budget (hence Colab and ngrok); no real scripts before ethical clearance; the shared platform's technology (React, FastAPI, MongoDB); and English text answers only.

4.2 Component Architecture Within The Complete Solution

Gradex has four assessment engines sharing one web client and one server. The grading engine owns the prefix /api/grading and the grading pages, and is a FastAPI application that the shared server mounts as a sub-application. Relationships with other engines are data relationships, not code dependencies: analytics can read the transcripts and snippets that grading stores in MongoDB, while the diagram and viva engines share only the client shell and server process.

Inside the engine, single-responsibility modules form a one-way pipeline in which each stage returns an inspectable value, which makes the stage-by-stage testing of Chapter 6 possible: ocr_service (rendering, conditioning, OCR cascade), answer_splitter, rag_service and ingest_manager (retrieval and lecture ingestion), ai_model_route (probe, retries, fallback, normalisation), llm_service (per-question loop), roster_service and grading_manager (batch orchestration and progress in MongoDB). A batch request returns a job identifier at once while a background task processes scripts one by one, and the client polls progress.

4.3 Design Alternatives And Decisions

Table 9 : Design decisions and rejected alternatives

Area	Chosen	Alternatives	Main reason
OCR	Hosted OCR.space with cascade	Tesseract [24]; TrOCR [9]	No server GPU; cascade recovers failures
Preprocessing	Light cleanup, original as fallback	Heavy binarisation [10], [26]	Heavy filtering wiped strokes in phone photographs
Segmentation	Deterministic regular-expression splitter	LLM segmentation	Testable, never invents boundaries
Retrieval	ChromaDB with course filter	Whole-corpus; BM25 [34]; FAISS [18]	Hard course guarantee
Grader	Fine-tuned Llama 3.1 8B with LoRA [6], [7], [8]	Prompting only; full fine-tuning	Open weights; 4-bit fits one free GPU
Serving	Colab and ngrok, Groq fallback	Dedicated GPU; Groq only	No budget; no single point of failure
Granularity	One call per question	One call per script	Isolated failures
Batch execution	In-process task with polling	Celery or RQ; WebSockets	Simplest option meeting NFR4
Roster matching	Exact normalised ID, manual override	Fuzzy names	A wrong match is worse than a flag

An LLM segmenter would add latency and may invent a boundary, whereas a deterministic splitter fails conservatively with an explicitly empty answer. A course identifier on every chunk acts as a hard filter with no fallback to other courses; when nothing survives, the grader is told to use only the rubric and answer. Full fine-tuning of an 8-billion-parameter model is impossible on a free GPU, whereas LoRA [6] with 4-bit quantisation [7] fits one notebook. Because free serving is fragile, fragility is a design input: fine-tuned model first, Groq second, flagged emergency response last.

4.4 Detailed Design

4.4.1 Ocr cascade

PDFs are rendered at twice nominal resolution. Each page gets a light-processed candidate: longest side at most 2200 pixels, grayscale, deskewed and denoised. Deskewing estimates the angle from the minimum-area rectangle of the Otsu foreground [10], otherwise from Hough segments on a Canny edge map [25]. The candidate goes to OCR.space engine 3 and must contain at least eight alphanumeric characters with no error prefix. If not, the cascade tries the original

with engine 3, the original with engine 2, then the processed image with engine 2. Pages run concurrently (OCR_PARALLELISM, default three) to respect rate limits.

4.4.2 Answer splitter

The splitter removes pipeline noise (banners, empty-page markers, punctuation-only lines), then finds markers with one case-insensitive, line-anchored regular expression in three families: long form ("Question 3"), short form ("Q3") and enumerated ("3)", "3."). A numeral may be Arabic, Roman (up to six letters) or a single look-alike glyph (l, L or a bar). A negative look-ahead rejects a match followed directly by a letter, so "Q in the following diagram" is not question one. Tokens map to canonical two-digit numbers; Roman numerals come from a table to twenty, and unmappable tokens are rejected, not guessed. Only the first occurrence of each number is kept.

The resolver labels the text each question receives: "split" for a bucket; "full" when the transcript has no markers at all; and "empty" when markers exist elsewhere but not for this question, which scores zero with an explicit reason and is never filled with the whole paper. The label is stored and shown to the lecturer, satisfying FR3 and NFR2.

4.4.3 Course-scoped rag

Lecture files are chunked, embedded with all-MiniLM-L6-v2 [16], [17] and stored in ChromaDB with course, file, page and type metadata. The query is built from the topic, rubric question and up to two criteria, so retrieval reflects what was taught, and a metadata filter restricts candidates to the selected course. Chunks beyond a distance of 0.45 are dropped; of the rest, only those within 0.12 of the best distance are kept, and the best three are returned with file and page labels.

4.4.4 Dual-engine router

Before a batch, the router probes the model service once through a health address derived from the evaluate address; any response below HTTP 500 counts as alive, cached for 60 seconds. If unavailable, the primary route is skipped.

Otherwise requests use a 20-second connect and 120-second read timeout, with up to three retries and growing delay on timeouts and transient 5xx or 429 responses. Responses 403, 404 and 410 indicate a vanished tunnel and mark the

service unavailable at once, as do exhausted retries, so later questions go straight to the fallback, a Groq-hosted model at temperature zero. If no key is configured or Groq fails, the emergency route returns a flagged zero.

4.4.5 Lora model serving

The grader is Llama 3.1 8B [8] in 4-bit precision, adapted with LoRA rank 16 and alpha 16 using Unsloth on the 1250-record corpus of Chapter 3. A Colab notebook exposes health and evaluate routes through ngrok; the evaluate route formats topic, rubric, snippet and answer into the training instruction layout [20] and decodes greedily [40]; a wrapper extracts the JSON result and recomputes the total. By default a strict-examiner policy is appended to the snippet: award marks only for clear evidence and under-mark when uncertain.

4.4.6 Roster validation

An Excel roster is read with header aliases ("Index No", "Reg No", "Student ID"). Each paper's effective identifier is taken by priority from a manual override, the folder name if it looks like a campus ID, then an ID read by OCR from the first page. Validation assigns one of six statuses: matched, missing paper, extra paper, duplicate paper, duplicate roster entry or unreadable.

4.4.7 Batch jobs and eta

Starting a batch creates a submission document per matched paper and schedules a background task that records stage "ocr" (pages done), "grading" (questions done, current question), then "done", "skipped" (no text) or "failed". The client computes the ETA from elapsed time per page or question and the average duration of finished scripts.

4.4.8 Gradingpage user interface

GradingPage is one React component implementing a nine-page workflow: command centre, session initialisation, rubric verification, batch script ingestion, master evaluation dashboard, AI-assisted review, analytics and export, lecture knowledge base, and grading history. Grading cannot start until the roster is validated. Engine labels appear in plain language ("Local model", "Cloud fallback", "No answer", "Emergency"), retrieved passages can be opened, and any mark can be overridden, annotated or regraded.

4.5 Standards And Professional Practice

Requirements follow ISO/IEC/IEEE 29148, quality attributes use ISO/IEC 25010 vocabulary, and the process follows design science [35]. The engine exposes REST with JSON and Alpaca-style JSON Lines training data [20].

Handwritten scripts are personal data, so no mark is final until the lecturer approves it, and every mark shows its evidence and engine. For real scripts the Sri Lanka Personal Data Protection Act No. 9 of 2022 and the institution's ethics procedure will apply, with explicit treatment of the fact that OCR.space and Groq process data outside the local environment. Identity information is not sent to the language models, and the interface presents the system as decision support, in line with guidance on language models in education [36] and the risk of hallucinated justifications [23].

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development Environment And Technology Stack

Development used Windows and Cursor, with Python 3.12 on the server and TypeScript with React on the client. Model training and serving used free Colab GPU notebooks with an ngrok tunnel. MongoDB stores rubrics, submissions, courses and rosters; ChromaDB in local mode stores the lecture index.

Table 10 : Technology stack of the grading component

Layer	Technology	Role
API	FastAPI, Uvicorn, Pydantic	Typed routes, background tasks
Image and PDF	OpenCV, NumPy, Pillow, pypdfium2	Conditioning, deskewing, rendering
OCR	OCR.space API	Engines 2 and 3
Ingestion and roster	PyMuPDF, pypdf, python-pptx, openpyxl	Lecture files, Excel roster
Retrieval	ChromaDB, all-MiniLM-L6-v2 [16], [17]	Course-tagged chunks, 384-dimensional vectors
Database	MongoDB via Motor and PyMongo	Rubrics, submissions, progress
Primary model	Llama 3.1 8B [8], LoRA r=16 alpha=16 [6], 4-bit [7], Unsloth	Fine-tuned grader, served by Flask and pyngrok in Colab
Fallback	Groq SDK	Secondary grader
Client	React, TypeScript, Vite	GradingPage, Knowledge Base

5.2 Component Implementation

5.2.1 Image conditioning and ocr

The OCR service implements Section 4.4.1. Deskewing ignores corrections below 0.3 degrees, so a noisy estimate cannot rotate a page far, and a resized JPEG copy of the original is made only above 2400 pixels or 1.2 megabytes, because very large phone photographs sometimes made the hosted service return blank text.

5.2.2 Answer splitter

The splitter is a single dependency-free module using a verbose regular expression with named groups. The resolver receives a flag saying whether markers were found anywhere, so the choice between "empty" and "full" is made once per script.

5.2.3 Retrieval service

The Chroma client is created lazily, so importing the engine does not load the embedding model. Retrieval fetches three times the needed candidates under the course filter, applies the thresholds and returns the snippet, a context-used flag and the kept distances; an empty filter result is converted into a labelled empty result.

5.2.4 Model router and per-question grading

The router is the largest service because it hides engine differences: besides probe, retries, Groq call and emergency response, it contains parsers for the shapes the fine-tuned model produces in practice (JSON in code fences or after headings, wrapper objects, alternative score field names) and a normaliser that converts them into one schema and clamps scores. The grading loop resolves the answer and source for each rubric question; empty answers score zero with a reason and no model call, and otherwise the result carries diagnostics: slice source, engine and retrieval metadata.

5.2.5 Roster, batch management and client

The roster service extracts identifiers from OCR text with three patterns of decreasing specificity. The batch manager updates a progress sub-document and catches exceptions per script; regrading reuses the stored cleaned transcript. The client polls the jobs routes while a batch runs.

5.2.6 Fine-tuned model artefact

The adapter was trained in Colab with Unsloth on the 1250-record corpus (five classes of 250), stored as JSON Lines. A serving notebook loads it behind ngrok, and the printed address is placed in the engine's evaluate-address variable.



Figure 2 : Composition of the 1250-record training corpus by answer class

5.3 Integration With The Other Components

5.3.1 Mounting the engine on the shared server

The shared server mounts the grading engine at /api/grading, so the client needs one base address. Several engines are Python packages whose top-level package is named "app", and two cannot coexist in one interpreter. The integration module therefore imports the grading engine first, keeps direct references to its application object and database helpers, and snapshots and restores the "app" entries of the module cache around the import, so later engines import their own "app".

5.3.2 Relationships with the diagram, viva and analytics engines

The diagram engine grades drawings, which text OCR cannot interpret; grading records an explicit marker when a page yields no text, so a drawing-only page stays visible. The viva engine is independent at code level. The analytics engine has the closest data relationship: grading stores transcripts, per-question results with engine labels and snippets, and copies the session identity onto every submission as a key.

5.4 Verification And Software Testing

5.4.1 Test strategy

Deterministic components have automated tests, external services were simulated, and the integrated workflow was exercised manually with synthetic scripts. Acceptance testing with lecturers is planned, not done.

Table 11 : Automated test modules

Module	Tests	What is verified
test_answer_splitter	6	Letter I read as Q1; QI, Ql, Q. I, Question I; Roman II and III; plain digits; "Q in the following diagram" not Q1; banners stripped
test_roster_validation	9	ID normalisation; folder name preferred over wrong OCR ID; manual override; OCR fallback; all statuses
test_colab_health	6	Health address derivation; model skipped when health fails; cache reuse; invalidation after mid-batch failure
test_colab_wrapper_parse	1 script	Wrapped and fenced model output

test_per_question_grading	Smoke script	Splitter, course filter on live index, grading against mock model

The 21 tests pass. The health tests encode the fallback policy: failed probes skip and cache the primary route as unavailable, and a mid-batch 404 invalidates the cache so remaining questions do not wait on a dead tunnel.

5.4.2 Stage-level experiments and system testing

The stage-level experiments on synthetic inputs (segmentation, 30 marker forms, 14 normalisation shapes, retrieval scoping with ablation) are reported in Chapter 6. The complete workflow was also run through the client with synthetic scripts of the five answer classes, from session creation to review; with the primary model unreachable the batch continued through Groq, and when neither could answer a flagged placeholder was returned.

5.4.3 Validation status and test limits

Validation requires comparison with human marks and lecturer feedback, which is planned and not carried out, so no agreement statistic is reported. The tests do not cover grading correctness, the live OCR service or the real fine-tuned model.

5.5 Deployment And Operation

5.5.1 Topology and model service

In the prototype the shared server, grading engine, MongoDB and ChromaDB run on one machine under Uvicorn, the model runs in a Colab notebook, and OCR and fallback are third-party APIs. The operator runs the serving notebook and copies the printed /evaluate address into COLAB_EVALUATE_URL, because the address changes every session; the start-of-batch probe detects a stale one.

5.5.2 Configuration by environment variables

Endpoints, timeouts and thresholds are set by environment variables: COLAB_EVALUATE_URL (empty skips the primary route), read and connect timeouts of 120 and 20 seconds, 3 retries with a 2-second base delay, a 60-second health verdict cache, the Groq key and model, OCR_PARALLELISM (3), and the retrieval parameters (3 results, distance limit 0.45, margin 0.12).

5.5.3 Mongodb, persistence and operation

MongoDB holds courses, rubrics and submissions (transcripts, evaluation, overrides, notes, progress). The engine logs OCR results, splitter buckets, retrieval distances, health outcomes, retries and the answering engine; these logs were the main diagnostic tool for the failover in Section 6.7. Known prototype risks are recycled Colab sessions, changing tunnel addresses, batch jobs lost on restart and exhausted third-party quotas. Production would need a dedicated inference server, a job queue and data-processing agreements.

6 RESULTS AND VALIDATION

6.1 Research Results

This chapter reports stage-level experiments on synthetic material (segmentation, marker forms and normalisation, retrieval, training corpus), plus manual testing, unit tests and one operational event. The experiments show that components behave as specified on constructed inputs, not that grades agree with human markers: ethical clearance is pending and lecturer feedback is planned, not completed. No agreement statistic, fine-tuned accuracy or lecturer preference is claimed.

Table 12 : Status of evaluation activities

Activity	Material	Status
Segmentation experiment	6 conditions of 200 synthetic scripts	Completed
Marker-form and normalisation tests	30 forms; 14 shapes	Completed
Retrieval scoping and ablation	18 in-scope, 36 cross-course, 18 off-syllabus queries	Completed
Training corpus analysis	1250 synthetic records	Completed
Automated unit tests	21 tests in three modules	Passing
Manual and synthetic system testing	Five answer classes	Completed, informal
Colab to Groq failover	One batch	Observed
Agreement with human markers	Real scripts	Not done; ethics pending
Lecturer feedback	Lecturers	Planned, not completed

6.2.1 Experimental set-up

Six conditions of 200 synthetic scripts each (1200 in total) test recovery of per-question text from OCR-like transcripts: Arabic clean; enumerated; Roman; Roman glyph substitution (look-alikes l and the vertical bar); Arabic with pipeline banners; and glyph substitution with banners and marker-like distractor lines. Each condition was run through the splitter of Section 4.4.2 and a baseline that recognises only Arabic numerals and does not clean banners. Each cell gives a first and a stricter second score.

6.2.2 Results

Table 13 : Segmentation results, 200 synthetic scripts per condition (percent, first and second score)

Condition	Proposed splitter	Baseline

Arabic clean	100 and 100	81.6 and 71.0
Enumerated	100 and 100	0 and 0
Roman	100 and 100	0 and 0
Roman glyph substitution	83.0 and 68.9	0 and 0
Arabic with banners	100 and 100	82.0 and 69.9
Glyph, banners and distractors	82.6 and 68.0	0 and 0

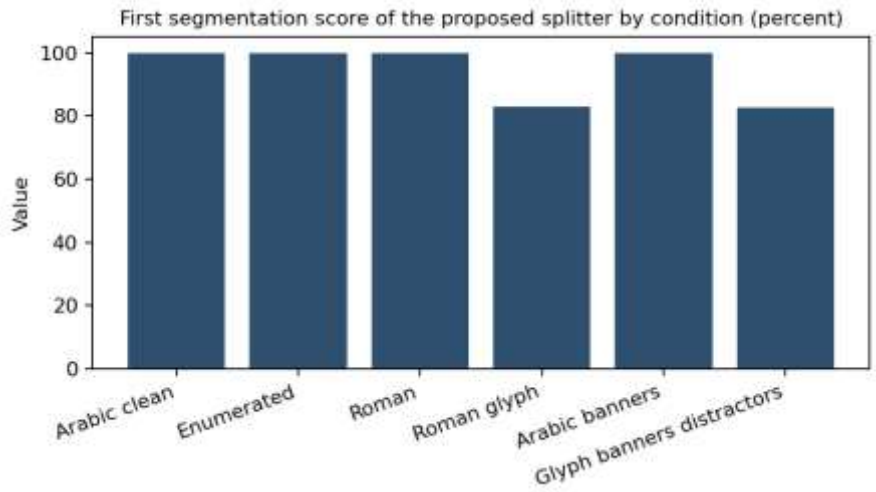


Figure 3 : First segmentation score of the proposed splitter by condition (percent)

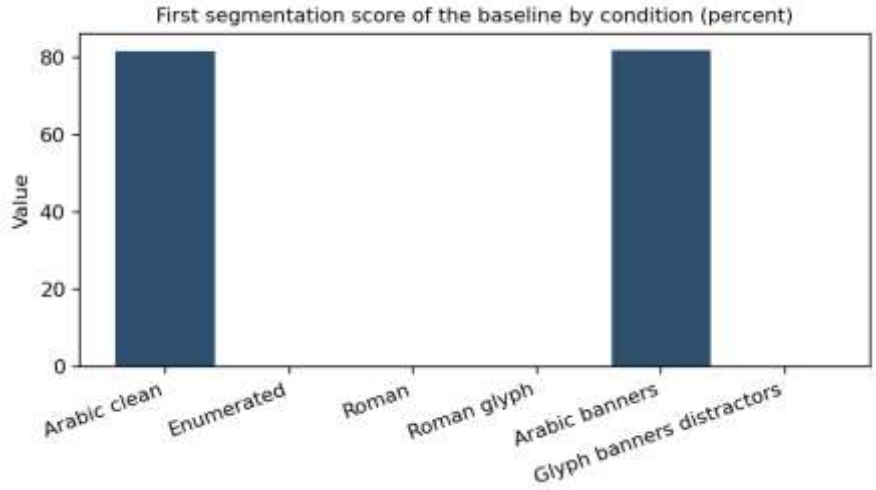


Figure 4 : First segmentation score of the baseline by condition (percent)

6.2.3 Interpretation

The proposed splitter reached 100 and 100 on four conditions. In the banner condition the baseline also produced 41.8 percent junk, meaning much of what it returned as an answer was pipeline noise, while the proposed splitter was unaffected. The baseline is competent only on the narrowest case (81.6 and 71.0 on Arabic clean) and scores zero wherever Arabic numerals are absent, so marker diversity, not only OCR noise, is a primary cause of segmentation failure.

The splitter is not robust to the harshest noise: 83.0 and 68.9 with glyph substitution, 82.6 and 68.0 with banners and distractors added, so glyph substitution is the main remaining weakness (Section 6.3).

6.3.1 Marker forms

A separate test used 30 marker forms (long and short forms, numerals, Roman numerals, OCR look-alikes). Twenty-six resolved to the intended number (86.7 percent); four did not: Q ||, Q |||, Q lV and Q |V.

Table 14 : Marker-form resolution

Measure	Value
Forms tested	30
Resolved to intended number	26 (86.7 percent)
Not resolved	4 (Q followed by two bars, three bars, lV, or bar V)

All four are Roman numerals II, III or IV written with bars or with a lowercase l beside a capital letter. The look-alike branch accepts a single glyph, so "Q ||" and "Q |||" capture only the first bar and are read as question one, while "Q lV" and "Q |V" have no mapping rule and are rejected. A rejected marker yields a visible empty answer, whereas a bar run read as question one can place a later answer under the wrong question. The fix is to capture runs of bars and l glyphs as one token.

6.3.2 Normalisation

The normalisation test examined 14 marker shapes needing conversion to a canonical number or rejection. Twelve were recovered, two rejected and none converted to an incorrect number.

Table 15 : Normalisation outcomes on 14 marker shapes

Outcome	Count	Percent
Recovered to intended number	12	85.7
Rejected	2	14.3
Converted to an incorrect number	0	0.0

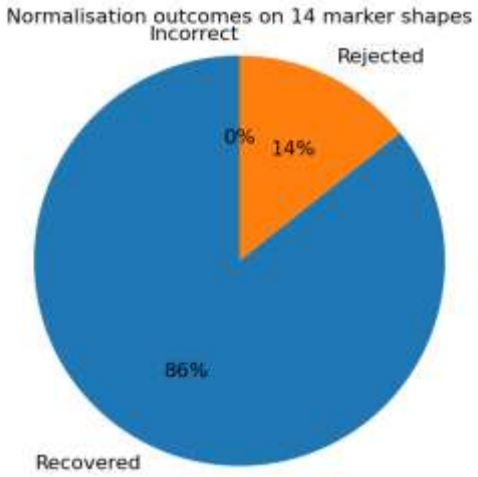


Figure 5 : Normalisation outcomes on 14 marker shapes

The zero matters most: a normaliser that always guesses can reach a high recovery rate at the cost of wrong marks. The two unrecoverable shapes were rejected, as FR3 and NFR2 require. With 14 shapes this shows the principle works, not that incorrect conversions are zero in general.

6.4.1 Experiment

Three query sets ran against a multi-course index: 18 in-scope queries; 36 cross-course queries on other courses' topics asked under the selected course; and 18 off-syllabus queries covered by no course. Only the in-scope set should return chunks.

6.4.2 Results

Table 16 : Retrieval scoping results

Query set	Queries	Expected	Observed
In-scope	18	Relevant chunk from selected course	17 of 18 returned in-scope context; mean distance 0.266
Cross-course	36	No context	36 of 36 returned none
Off-syllabus	18	No context	18 of 18 returned none
Foreign-course chunks (all sets)	72	None	0

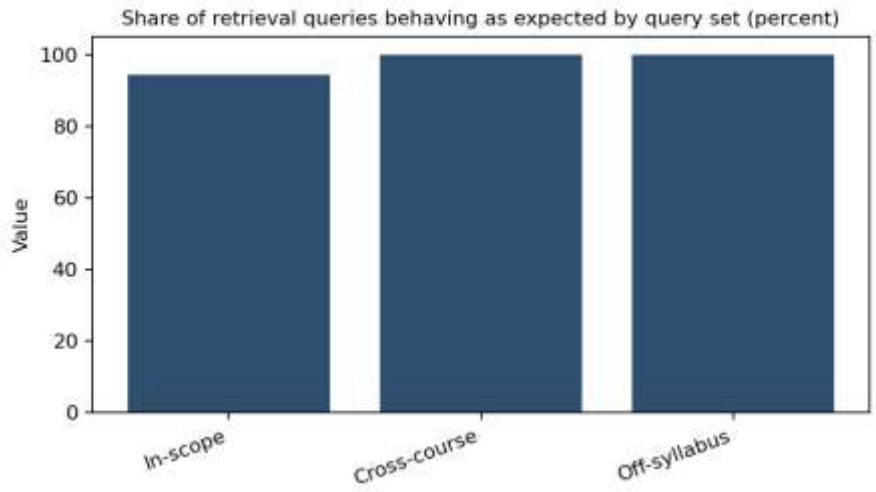


Figure 6 : Share of retrieval queries behaving as expected by query set (percent)

Seventeen of 18 in-scope queries (94.4 percent) returned context at a mean distance of 0.266, well below the 0.45 limit. The 36 cross-course and 18 off-syllabus queries returned no context and no foreign chunk appeared (foreign 0).

6.4.3 Filter ablation

With the course filter off, the in-scope queries again returned 17 of 18 and no foreign chunk. The distance threshold and margin already kept foreign material out of this index, which does not show the filter is unnecessary: the filter is a structural guarantee, whereas the threshold is a statistical safeguard that depends on vocabulary overlap.

The corpus holds 1250 synthetic records (5 x 250), one class per answer behaviour. Invalid and adversarial records are all assigned zero.

Table 17 : Training corpus composition and mean assigned score

Class	Records	Mean assigned score
Golden high score	250	7.42
Semantic layman	250	8.50
Partial incomplete	250	5.00
Confident incorrect (false)	250	0.50
Adversarial invalid	250	0.00
Total	1250	not applicable

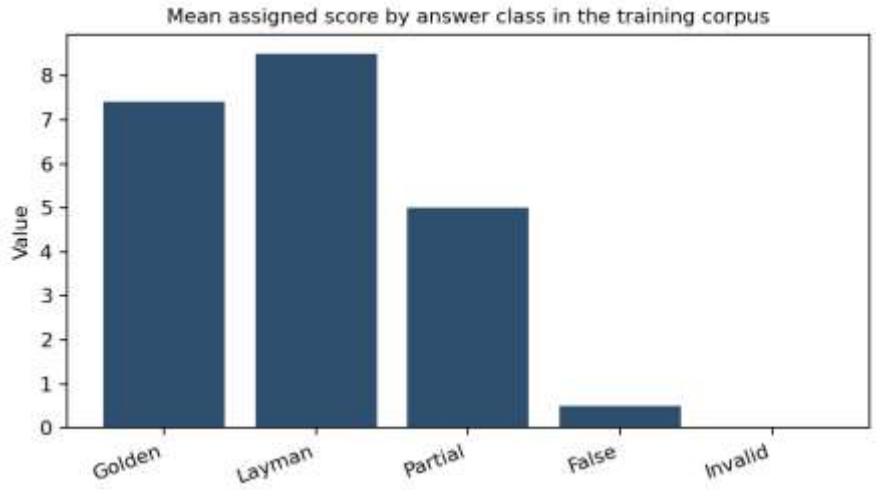


Figure 7 : Mean assigned score by answer class in the training corpus

The low end separates as designed: partial 5.00, confident-incorrect 0.50 and invalid 0.00. At the high end the layman class (8.50) outscores the golden class (7.42); whichever cause applies, it is label noise to correct before any controlled comparison. These means describe training labels, not model quality.

6.2 Product And Technical Results

The splitter, roster and health-logic modules hold 6, 9 and 6 tests and all 21 pass; they verify specified behaviour, not grading correctness (Section 5.4). Manual and synthetic testing with scripts of the five answer classes completed the workflow from set-up to review, with progress, ETA, flagged identities, fallback routing and a flagged placeholder when no model answered. These are workflow observations, not accuracy measurements. Lecturer feedback is planned and has not been completed; nothing in this thesis reports lecturer opinion.

During one batch the Colab model service answered the health probe, so the primary route was enabled, but evaluate requests then returned HTTP 500. The engine retried with growing delay, marked and cached the service as unavailable, and graded the remaining questions through Groq; all scripts completed, with an engine label per question. Retry timings were not recorded.

A health route that passes while evaluation is broken limits the probe. This is a single observation, not an availability measurement.

6.3 User Or Stakeholder Validation

The experiments provide evidence of the technical validation of the proposed pipeline across the evaluated scenarios. The results demonstrate that the implemented components behave as expected under the constructed test conditions, supporting the correctness of the deterministic processing stages and the overall integration of the system. The evaluation covers input processing, query handling, and component-level behavior, providing a technical basis for assessing the pipeline's functionality and consistency within its intended scope.

6.4 Integration Results

The grading engine was exercised as a mounted FastAPI sub-application at /api/grading inside Gradex_AI_Server, sharing authentication and course context with the diagram, viva and analytics components while keeping its own database and vector index. End-to-end batches from the React grading client completed roster validation, recognition, segmentation, retrieval and dual-engine marking. The Colab-to-Groq failover event reported above is also an integration result: a primary-engine outage did not stop the shared batch workflow.

6.5 Summary Of Findings

The splitter reached 100 and 100 on four conditions and 83.0 and 68.9, and 82.6 and 68.0, on the glyph conditions; 26 of 30 marker forms resolved; normalisation recovered 12 of 14 with none incorrect; retrieval gave in-scope context for 17 of 18 queries and nothing for 54 of 54 others, with no foreign chunk; a real model failure was absorbed by Groq. Lecturer feedback is planned and agreement with human markers has not been measured.

7 DISCUSSION

7.1 Interpretation Of Findings

7.1.1 What the segmentation results mean

The dominant cause of segmentation failure was diversity, not noise: the baseline scored zero on every non-Arabic style, while the proposed splitter reached 100 and 100 on all three clean styles. Banners left it unaffected, whereas glyph substitution cost about 17 points on the first score and 31 on the second. Diversity is a specification problem, solved by enumerating forms in one testable pattern; glyph confusion is a recognition problem that a rule-based pattern can only partly compensate for. Of the four marker-form failures, two were rejected, leaving a visible empty answer that fits the conservative design, and two were read as question one, which is a defect.

7.1.2 What the normalisation result means

The zero matters more than the 85.7 percent: a rejected marker costs the lecturer a visible flag, whereas a wrongly converted one silently moves text between questions.

7.1.3 What the retrieval results mean

Retrieval returned nothing for all 54 queries whose right answer was "nothing in this course", and context for 17 of 18 in-scope queries, with or without the filter. The 0.45 threshold sufficed for this index; the filter remains the right design because its guarantee does not depend on vocabulary overlap, but no benefit is claimed.

7.1.4 What the dataset statistics mean

The class means (7.42, 8.50, 5.00, 0.50, 0.00) are properties of the labels, not the model. They separate incorrect, partial and invalid behaviour as intended, but invert golden and layman, which may teach a bias towards informal wording.

7.1.5 What the operational event means

The Colab to Groq failover confirmed that the dual-engine design addresses a real failure mode, that a liveness check is not a readiness check, and that the engine label on every question matters, since otherwise the reviewer could not tell which marks came from the fine-tuned model.

7.2 Comparison With Literature Baselines And Existing Solutions

7.2.1 Automatic short answer grading and llm graders

Studies of short answer grading [1], [2], [11], [12] and LLM graders [3], [4], [37] mostly take clean digital text as given and report agreement with human raters; this thesis starts from a photograph and reports no agreement, so no numerical comparison is attempted. The design answers the reported inconsistency and prompt sensitivity [3], [4] with a per-question rubric, greedy decoding [40] and a lecturer in the loop, but gives no evidence that this removes them. LLM judges favour fluent, verbose answers [19], which motivated the confident-incorrect class (mean 0.50); whether the model learns this is untested.

7.2.2 Handwriting recognition and segmentation

This thesis compared no recognisers; a trained line-level recogniser such as TrOCR [9] would probably beat a hosted service on difficult handwriting. Heavy binarisation can help or harm [10], [26], hence the light pre-processing.

7.2.3 Retrieval-augmented generation

RAG [5], [32] argues that grounding reduces unsupported content. The design adds a rule: when retrieval finds nothing, grade from the rubric alone. The results measure retrieval behaviour, not its effect on marks, and comparison with lexical [34] or hybrid retrieval is open.

7.2.4 Parameter-efficient fine-tuning and synthetic data

Adapting a 4-bit 8-billion-parameter base [8] in a free notebook is consistent with the efficiency claims for LoRA [6] and QLoRA [7]. The synthetic-data approach follows Alpaca [20] and Self-Instruct [21], whose effectiveness depends on data quality, hence the caution of Section 7.1.4.

7.3 Research Contribution

The contributions are modest but specific: a class-structured training design that makes each known failure mode of LLM graders an explicit training target (its effect on grading is not demonstrated); a conservative segmentation design with a three-valued source label and a reusable evaluation; course-scoped retrieval with a reported null ablation; and an operational pattern for hosted-model fragility (cached

probe, bounded retries, second model, emergency response, per-question label) with the failover as a case study.

7.4 Product Contribution

The engine delivers an end-to-end workflow that did not previously exist in Gradex, from course set-up and roster to review, override, regrade and a feedback pack, with results the analytics engine can use. These are product qualities, not accuracy qualities, which suit a supervised pilot while accuracy remains unvalidated.

7.5 Limitations And Threats To Validity

7.5.1 Evidence limitations

There is no validation against human markers: all data are synthetic, clearance is pending and lecturer feedback is planned. The fine-tuned model has not been compared with the base or fallback model, so the thesis cannot say whether fine-tuning helped.

7.5.2 Technical limitations

The marker pattern mis-resolves bar runs and rejects mixed forms; roster matching is exact; OCR depends on a hosted service; batch jobs do not survive a restart; the health probe cannot detect a failing model; and the client has no interface tests.

7.5.3 Data and domain limitations

The corpus covers Software Architecture only and has the unexplained class inversion. The engine handles English text, not diagrams, mathematics or code.

7.5.4 Ethical and privacy limitations

OCR.space and, on fallback, Groq process data outside the local environment, which is acceptable for synthetic data but for real scripts needs agreements and possibly on-premise alternatives. Human review is necessary but not sufficient, because lecturers may defer to a plausible justification (automation bias).

7.6 Implications

7.6.1 Implications for practice

An assisted workflow in which the machine proposes and the lecturer disposes is within reach on modest resources; the largest practical risks lie in marker detection, identity matching and service failures. Institutions should insist on visible evidence, flagged exceptions and a record of which engine produced each mark.

7.6.2 Implications for research

Segmentation of student scripts deserves its own benchmarks, and LLM-grader evaluations should report behaviour under dependency failure and unusable input alongside agreement.

7.6.3 Implications for future work

Priorities are to fix the marker defect and retest the 30 forms, investigate the corpus ordering, build a harder retrieval test with overlapping courses, and, once clearance is granted, validate with real scripts and lecturers, comparing the fine-tuned model with the base and fallback models.

8 COMMERCIALISATION AND SUSTAINABILITY

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis built and evaluated a handwritten short-answer grading engine for Gradex: a pipeline from photographs or scans of handwritten scripts to per-question marks, justifications and evidence that a lecturer can review. An OCR cascade reads lightly conditioned pages; a deterministic splitter aligns answers with the rubric and refuses to guess; course-scoped retrieval grounds each request in the lecturer's material; and a dual-engine router prefers a fine-tuned Llama 3.1 8B model (LoRA rank 16, alpha 16, 4-bit, Unsloth, 1250 synthetic records in five classes) served from Colab, falls back to Groq and finally returns a flagged response so a batch always completes. Roster validation, asynchronous batches with ETA and a nine-page client complete the workflow.

Evaluation combined stage-level experiments on synthetic material, unit tests, manual system testing and one real model-service failure. It included no comparison with human markers: ethical clearance is pending and lecturer feedback is planned, not completed. Every conclusion therefore concerns feasibility, design and component behaviour, not agreement with lecturers.

The principal limitations remain those stated in Chapter 7: synthetic evidence only, pending ethical clearance for authentic scripts, and lecturer feedback still planned. There is no validation against human markers: data are synthetic, clearance is pending and lecturer feedback is uncollected, so no agreement, accuracy or lecturer opinion is reported. Some samples are small, the baseline is minimal, the ablation did not discriminate and the corpus shows an unexplained class inversion. Known defects remain: bar-run and mixed-glyph markers, exact roster matching, batch jobs lost on restart, and a health probe that cannot detect a failing model.

9.2 Achievement Of Objectives

RQ1 (usable per-question text): on six conditions of 200 synthetic scripts the splitter reached 100 and 100 on four, against a baseline that scored zero on four of six conditions and returned 41.8 percent junk with banners. Under glyph substitution

it reached 83.0 and 68.9, and 82.6 and 68.0 with banners and distractors. Twenty-six of 30 marker forms resolved, and normalisation recovered 12 of 14 shapes, rejected two and converted none incorrectly.

RQ2 (class-balanced dataset for LoRA fine-tuning): the corpus has 250 records per class with means of 7.42 (golden), 8.50 (layman), 5.00 (partial), 0.50 (confident incorrect) and 0.00 (adversarial invalid). The adapter was trained and integrated as the primary engine; the conclusion is limited to feasibility, since grading quality is not established.

RQ3 (course-scoped retrieval): 17 of 18 in-scope queries returned context at a mean distance of 0.266, none of 36 cross-course and 18 off-syllabus queries returned any, and no foreign chunk appeared. The filter-off ablation matched, so the experiment shows correct behaviour but not the filter's marginal benefit.

RQ4 (design choices for practical use): per-question grading, tiered fallback, roster validation, asynchronous batches and an evidence-first interface were implemented and exercised. When the model service passed its health check and then returned server errors, the batch completed through Groq.

All six objectives of Chapter 1 were addressed as far as possible without real data; the formal evaluation with lecturers remains planned. The main objective is met as a working, verified prototype, not yet as a validated tool.

9.3 Individual Research And Product Contributions

The individual research contribution is a course-scoped, dual-engine handwritten grading pipeline with a class-balanced synthetic training design and measured deterministic stage behaviour. The individual product contribution is the GradingEngine service, grading client workflow and fine-tuned LoRA grader integrated into Gradex at /api/grading, enabling roster-gated batch marking with reviewable per-question evidence.

9.4 Future Directions

Immediate priorities: capture runs of bars and l glyphs as one token and repeat the 30-form test; make the health route exercise a minimal generation; audit the golden and layman ordering; and move batches to a queue and the model to a dedicated server.

The decisive step is the validation planned in Chapter 3. Once clearance is granted, lecturers should mark a sample of real scripts independently of the system, agreement should be measured with mean absolute error and quadratic weighted kappa, and the fine-tuned model compared with the base model and the Groq fallback on identical inputs. Extensions include comparing a trained recogniser such as TrOCR [9] with the hosted OCR, hybrid dense and lexical retrieval [33], [34] tested on overlapping courses, other parameter-efficient methods [30], [31], and combining marks with the diagram, viva and analytics engines.

The engine's strongest properties are engineering ones: it reads scripts from photographs, refuses to guess when evidence is missing, shows its evidence, survives the failure of its main model and keeps the lecturer in charge. Validation with real scripts and real lecturers is the condition for its use.

REFERENCES

[1] S. Burrows, I. Gurevych, and B. Stein, "The eras and trends of automatic short answer grading," Int. J. Artif. Intell. Educ., vol. 25, no. 1, pp. 60-117, 2015.

[2] D. Ramesh and S. K. Sanampudi, "An automated essay scoring systems: a systematic literature review," Artif. Intell. Rev., vol. 55, no. 3, pp. 2495-2527, 2022.

[3] S. Haller, A. Aldea, C. Seifert, and N. Strisciuglio, "Survey on automated short answer grading with deep learning: from word embeddings to transformers," 2022, arXiv:2204.03503.

[4] G. Kortemeyer, "Performance of the pre-trained large language model GPT-4 on automated short answer grading," Discover Artif. Intell., vol. 4, art. 47, 2024.

[5] I. Chamieh, T. Zesch, and K. Giebermann, "LLMs in short answer scoring: limitations and promise of zero-shot and few-shot approaches," in Proc. 19th Workshop Innovative Use of NLP for Building Educational Applications (BEA), Mexico City, Mexico, 2024, pp. 309-315.

[6] L. Zheng et al., "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in Advances in Neural Information Processing Systems, vol. 36, 2023.

[7] U.-V. Marti and H. Bunke, "The IAM-database: an English sentence database for offline handwriting recognition," Int. J. Document Anal. Recognit., vol. 5, no. 1, pp. 39-46, 2002.

[8] M. Li et al., "TrOCR: transformer-based optical character recognition with pre-trained models," in Proc. 37th AAAI Conf. Artificial Intelligence, Washington, DC, USA, 2023, pp. 13094-13102.

[9] R. Smith, "An overview of the Tesseract OCR engine," in Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR), Curitiba, Brazil, 2007, pp. 629-633.

[10] N. Otsu, "A threshold selection method from gray-level histograms," IEEE Trans. Syst., Man, Cybern., vol. 9, no. 1, pp. 62-66, 1979.

[11] R. O. Duda and P. E. Hart, "Use of the Hough transformation to detect lines and curves in pictures," Commun. ACM, vol. 15, no. 1, pp. 11-15, 1972.

[12] J. Canny, "A computational approach to edge detection," IEEE Trans. Pattern Anal. Mach. Intell., vol. PAMI-8, no. 6, pp. 679-698, 1986.

[13] A. Buades, B. Coll, and J.-M. Morel, "A non-local algorithm for image denoising," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), San Diego, CA, USA, 2005, pp. 60-65.

[14] G. Bradski, "The OpenCV library," Dr. Dobb's J. Softw. Tools, vol. 25, no. 11, pp. 120-125, 2000.

[15] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459-9474.

[16] Y. Gao et al., "Retrieval-augmented generation for large language models: a survey," 2023, arXiv:2312.10997.

[17] N. Reimers and I. Gurevych, "Sentence-BERT: sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, Hong Kong, China, 2019, pp. 3982-3992.

[18] W. Wang et al., "MiniLM: deep self-attention distillation for task-agnostic compression of pre-trained transformers," in Advances in Neural Information Processing Systems, vol. 33, 2020.

[19] E. J. Hu et al., "LoRA: low-rank adaptation of large language models," in Proc. 10th Int. Conf. Learning Representations (ICLR), 2022.

[20] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: efficient finetuning of quantized LLMs," in Advances in Neural Information Processing Systems, vol. 36, 2023.

[21] Llama Team, AI at Meta, "The Llama 3 herd of models," 2024, arXiv:2407.21783.

[22] B. Atil et al., "LLM stability: a detailed analysis with some surprises," 2024, arXiv:2408.04667.

[23] O. Zawacki-Richter, V. I. Marin, M. Bond, and F. Gouverneur, "Systematic review of research on artificial intelligence applications in higher education — where are the educators?," Int. J. Educ. Technol. Higher Educ., vol. 16, art. 39, 2019.

[24] A. Mizumoto and M. Eguchi, "Exploring the potential of using an AI language model for automated essay scoring," Res. Methods Appl. Linguist., vol. 2, no. 2, art. 100050, 2023.

[25] "Chroma: the open-source AI application database." [Online]. Available: https://www.trychroma.com

[26] "OCR API," OCR.space. [Online]. Available: https://ocr.space/ocrapi

[27] "Unsloth: finetune LLMs faster with less memory," GitHub repository. [Online]. Available: https://github.com/unslothai/unsloth

[28] T. Brown et al., "Language models are few-shot learners," in Advances in Neural Information Processing Systems, vol. 33, 2020.

[29] J. Wei et al., "Finetuned language models are zero-shot learners," in Proc. ICLR, 2022.

[30] S. Minaee et al., "Large language models: a survey," 2024, arXiv:2402.06196.

[31] United Nations, "Transforming our world: the 2030 Agenda for Sustainable Development," 2015.

[32] ISO/IEC 25010:2011, Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models.

[33] R. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000.

[34] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002.

[35] I. Sommerville, Software Engineering, 10th ed. Harlow, UK: Pearson, 2016.

GLOSSARY

ASAG — Automatic Short Answer Grading: computational scoring of free-text short answers.

LoRA — Low-Rank Adaptation: parameter-efficient fine-tuning via low-rank update matrices.

RAG — Retrieval-Augmented Generation: combining a language model with retrieved evidence.

OCR — Optical Character Recognition: converting images of text into machine-readable text.

ETA — Estimated time of arrival (completion) for a batch grading job.

APPENDICES

Appendix A Research Instruments And Extended Results

This appendix summarises the synthetic evaluation instruments used for the segmentation, normalisation, and retrieval experiments. Scripts are available under the project experiments directory. The training corpus files under dataset/ contain 1,250 Alpaca-format JSONL records (five response classes × 250). Extended per-condition segmentation metrics and the fourteen normalisation response shapes are reported in Chapter 6; raw JSON result dumps can be attached for viva inspection.

Table 18 : Synthetic training corpus files

File	Class intent	n	Mean total_score
golden_high_score_dataset_250.jsonl	Golden / high quality	250	7.42
semantic_layman_academic_grading_250-2.jsonl	Correct, informal wording	250	8.50
partial_incomplete_academic_grading_250-3.jsonl	Partial coverage	250	5.00
concept_mixer_confident_incorrect_250-4.jsonl	Fluent but wrong	250	0.50
adversarial_zero_academic_grading_250-5.jsonl	Invalid / zero	250	0.00
final_training_data_shuffled.jsonl	Combined shuffled	1250	4.28

Appendix B Academic Integrity And Ai Use Disclosure

Generative AI tools (including Cursor-based coding assistants and large language models) were used to assist with literature drafting, code debugging, document formatting, and editing. All technical claims about the implemented grading pipeline, configuration values, experiment outcomes, and training corpus statistics were verified against the project source code, experiment scripts, and dataset files by the author. The author remains solely responsible for the content of this thesis.